

Contents

1	Entorno Linux, Shell y reproducibilidad	1
1.1	Antes de la primera orden	1
1.2	Unix y sus paradigmas	2
1.2.1	Un poco de historia: el origen de la computación moderna	2
1.2.2	Por qué importa: presencia profesional e infraestructura científica	2
1.2.3	Filosofía de código abierto	3
1.2.4	Primer paradigma: todo es un fichero	3
1.2.5	Segundo paradigma: herramientas pequeñas que se combinan	3
1.3	El modelo terminal, Shell y procesos	3
1.3.1	El ciclo de vida de un proceso	4
1.3.2	Jerarquía y aislamiento de memoria	4
1.3.3	Privilegios: usuarios y superusuario root	5
1.3.4	El estado de salida de un proceso	5
1.4	El sistema de archivos: el mapa de navegación	5
1.4.1	Directorio de trabajo: el suelo donde pisamos	6
1.4.2	La brújula de la terminal	7
1.5	Navegación e inspección sin alterar	7
1.5.1	Inspección segura de datos biológicos masivos	8
1.5.2	La trampa de contar líneas frente a contar secuencias	9
1.5.3	El problema silencioso de los saltos de línea Windows (CRLF)	9
1.5.4	Ficheros ocultos y configuración	9
1.5.5	La verdad de su sistema: <code>-help</code> y <code>man</code>	9
1.6	Enlaces duros y enlaces simbólicos (<code>ln</code> y <code>ln -s</code>)	9
1.6.1	El valor de los enlaces simbólicos en bioinformática	10
1.7	Propiedad y permisos	10
1.7.1	El significado especial de la ejecución (<code>x</code>) en directorios	11
1.7.2	Modificación de permisos con <code>chmod</code>	11
1.8	Ejecutar un archivo y variables de entorno	12
1.8.1	La variable <code>\$PATH</code> frente a la ruta explícita <code>./</code>	12
1.8.2	El Shebang: el contrato de ejecución	13
1.8.3	Variables locales frente a variables de entorno exportadas	13
1.8.4	Variables predefinidas y configuración permanente	14
1.9	Ergonomía y productividad en la terminal	14
1.9.1	El historial y sus límites	14
1.10	Espacio de almacenamiento y organización del proyecto	15
1.10.1	Estructura canónica de un proyecto bioinformático	15
1.11	Antes de una operación que sobrescribe o elimina	15
1.12	Reproducibilidad, empaquetado e integridad	16
1.12.1	Empaquetado portable con <code>tar</code>	16
1.12.2	Integridad criptográfica con <code>sha256sum</code>	17

1.13 Método de verificación y errores frecuentes	17
1.14 Síntesis de referencia	18
1.15 Glosario esencial	19
1.16 Fuentes y límites de uso	19
Anexo práctico · Ambientarse, inspeccionar y entregar de forma reproducible	21

Chapter 1

Entorno Linux, Shell y reproducibilidad

1.1 Antes de la primera orden

El análisis masivo de datos biológicos rara vez cabe en una hoja de cálculo. Cuando un secuenciador de última generación entrega decenas de gigabytes con millones de lecturas genómicas, o cuando un modelo estructural predice miles de conformaciones proteicas, las interfaces gráficas convencionales colapsan o se vuelven impracticables. En ese momento, la pantalla negra de la terminal deja de ser una reliquia del pasado para convertirse en la herramienta más potente, rápida y elegante a disposición de un científico.

La terminal no es un entorno hostil diseñado para programadores expertos; es, en esencia, una conversación directa y sin intermediarios con el sistema operativo. En vez de hacer clic sobre botones que ocultan lo que ocurre por debajo, usted escribe instrucciones precisas que transforman flujos de información biológica de forma transparente y automatizable.

Sin embargo, en biomedicina y bioinformática, obtener un número o una tabla no basta. La ciencia exige que cualquier análisis ejecutado hoy pueda ser repetido fielmente por otra persona —o por usted mismo seis meses más tarde— y produzca exactamente el mismo resultado. A esta exigencia la llamamos **reproducibilidad computacional**.

Este capítulo establece los cimientos de todo su trabajo técnico posterior. Aprenderá cómo está construido el ecosistema Unix/Linux; qué ocurre en la memoria del ordenador cuando se ejecuta un programa; cómo orientarse en el sistema de archivos sin perderse; cómo inspeccionar ficheros biológicos masivos sin alterarlos; cómo gestionar permisos y enlaces; cómo personalizar su entorno de trabajo; y cómo registrar un experimento computacional para que sea inmutable y verificable.

La ruta **esencial** comprende la filosofía Unix, el modelo de procesos, el mapa del sistema de archivos, la navegación e inspección segura, los permisos, los enlaces simbólicos y la ejecución de programas; estimada en 90–120 minutos. La ruta de **estudio** añade variables de entorno, personalización permanente, ergonomía de terminal, monitorización de almacenamiento, registro de entorno reproducible, protocolo de seguridad ante operaciones destructivas y empaquetado verificable; 70–100 minutos adicionales.

Esencial

Antes de empezar · cuatro predicciones formativas. Sin ejecutar ninguna orden en la terminal, anote su predicción razonada ante estas cuatro situaciones:

1. ¿Designa siempre la expresión `datos/muestra.fasta` exactamente el mismo fichero?
2. ¿Un fichero con permisos de lectura es obligatoriamente ejecutable por el usuario?
3. ¿Un estado de salida 0 (`$? = 0`) certifica que el resultado científico del cálculo es correcto?

4. ¿Una variable creada en la sesión actual de Bash llega automáticamente a un subproceso hijo?

Anote mentalmente sus respuestas iniciales. Las fundamentaremos y resolveremos progresivamente en las Secciones 1.4, ??, ?? y ??, respectivamente.

1.2 Unix y sus paradigmas

1.2.1 Un poco de historia: el origen de la computación moderna

En 1969, en los laboratorios Bell de AT&T, Ken Thompson y Dennis Ritchie comenzaron a diseñar un sistema operativo interactivo, multitarea y multiusuario al que bautizaron como Unix. Poco después, en 1972, Ritchie creó el lenguaje de programación C expresamente para reescribir Unix. Esta decisión fue revolucionaria: hasta entonces, los sistemas operativos estaban atados al hardware específico para el que se programaban. Al estar escrito en C, Unix pudo compilarse y ejecutarse en máquinas de fabricantes muy diversos, convirtiéndose en el primer sistema operativo verdaderamente portable.

A partir de 1974, AT&T licenció Unix a universidades y centros de investigación a precios simbólicos. En la Universidad de California en Berkeley nació una influyente variante académica conocida como BSD (*Berkeley Software Distribution*). Años más tarde, en 1983, Richard Stallman fundó el proyecto GNU con el objetivo de construir un sistema operativo completamente libre y compatible con Unix. En 1991, un estudiante finlandés llamado Linus Torvalds desarrolló el núcleo (*kernel*) que faltaba para completar ese sistema: nacía Linux. Por su parte, la familia de sistemas macOS de Apple deriva directamente de NeXTSTEP (creado en 1988 tras la salida de Steve Jobs de Apple) y adoptó en 1996 Darwin, un núcleo de tipo Unix basado en BSD y Mach.

Unix se desarrolló en los laboratorios Bell de AT&T en 1969 (Ken Thompson y Dennis Ritchie); Linux y macOS son sistemas derivados o basados en Unix, no el propio Unix original.

"Unix" y "tipo Unix" (*Unix-like*) no son lo mismo: Linux y macOS clonan o derivan el funcionamiento central de Unix sin ser Unix en sentido estricto; el funcionamiento *core* de los sistemas basados en Unix es prácticamente idéntico entre ellos.

Esta genealogía no es una anécdota histórica: es la razón directa por la que los conocimientos de línea de comandos y Bash que aprenderá en este libro funcionan con idéntica precisión en un portátil con Linux, en un MacBook o en el nodo de cómputo de un superordenador internacional.

1.2.2 Por qué importa: presencia profesional e infraestructura científica

En la actualidad, los sistemas basados en Linux constituyen el estándar predominante de la computación científica y de la infraestructura informática mundial. No se trata de una preferencia estética, sino de una realidad operativa:

Los sistemas basados en Linux constituyen el estándar predominante en servidores, infraestructuras de cálculo científico en la nube y supercomputación, proporcionando el entorno operativo homogéneo donde se ejecutan los análisis bioinformáticos modernos.

La conclusión didáctica no depende de una cuota estadística puntual: cuando descargue datos del NCBI o de Ensembl, ejecute un modelo estructural con AlphaFold, alinee millones de lecturas en el clúster de un hospital o lance un contenedor reproducible, estará interactuando de forma invariable con un entorno Linux.

1.2.3 Filosofía de código abierto

El software de código abierto (*open source*) permite que cualquiera vea, modifique y redistribuya el código; su desarrollo suele ser descentralizado y colaborativo, con comunidades que revisan el código públicamente en repositorios (GitHub, GitLab, etc.).

En biomedicina, la ciencia abierta es un imperativo ético y metodológico. Si un algoritmo calcula una dosis terapéutica o predice la patogenicidad de una variante genética, la comunidad debe poder auditar el código fuente para entender su mecanismo y descartar artefactos. Linux, Bash y las utilidades que empleamos en este curso son software libre y transparente.

1.2.4 Primer paradigma: todo es un fichero

En el diseño clásico de Unix existe un principio unificador de enorme elegancia: *todo es un fichero* (*everything is a file*).

En los sistemas Unix, toda entrada o salida de un recurso de hardware se realiza a través de un fichero: un disco, la memoria, una conexión de red o una pantalla se manejan con el mismo tipo de herramientas y la misma interfaz homogénea que un documento de texto.

Este principio significa que para el sistema operativo no hay una diferencia conceptual abismal entre leer una línea de un archivo FASTA guardado en su disco duro, recibir bytes transmitidos por un secuenciador a través de la red o leer los datos que otro programa le está enviando en tiempo real: todos ellos se presentan como una corriente de bytes (*stream*) que puede abrirse, leerse, transformarse y cerrarse con las mismas órdenes básicas.

1.2.5 Segundo paradigma: herramientas pequeñas que se combinan

Frente a la tentación de construir programas gigantescos y monolíticos que intenten resolver todas las tareas posibles (y que acaban siendo frágiles, difíciles de mantener e incompatibles entre sí), Unix apostó por una arquitectura radicalmente modular: construir herramientas pequeñas y robustas, donde cada una haga una sola cosa excepcionalmente bien, y dotarlas de mecanismos estándar para conectarse entre sí.

Un ejemplo clásico es el comando **tar**: su única función es empaquetar múltiples ficheros dentro de un único contenedor sin comprimir. Cuando se necesita comprimir, Unix no reescribió **tar** para convertirlo en un compresor; utilizó **gzip**, un programa especializado en comprimir flujos de bytes.

El diseño de **tar -z** —integrar la compresión **gzip** directamente en el proceso de archivado sin fusionar ambos programas en uno— ejemplifica la filosofía Unix de herramientas pequeñas diseñadas para trabajar juntas de forma fluida, no de un programa único con todas las funciones.

En el capítulo siguiente (TC-CH02) aprenderá a encadenar estas herramientas mediante tuberías (**|**), construyendo auténticas cadenas de montaje de datos biológicos.

1.3 El modelo terminal, Shell y procesos

Para diagnosticar con éxito cualquier situación en la línea de comandos, es imprescindible distinguir tres elementos que los principiantes suelen confundir:

Una terminal es una interfaz de texto que envía órdenes a una Shell, la cual las interpreta y, cuando procede, lanza procesos; el resultado observable de un proceso incluye su salida y un estado de salida entero.

- **La terminal:** Es simplemente la ventana visual (el emulador) que captura lo que usted teclea y dibuja los caracteres que el sistema devuelve en la pantalla.

- **La Shell (el intérprete):** Es el programa que lee las líneas de texto que usted escribe, analiza su sintaxis, busca los ejecutables correspondientes y solicita al sistema operativo su ejecución. En este libro utilizamos **Bash** (*Bourne Again Shell*).
- **El proceso:** Es la instancia viva de un programa ejecutándose en la memoria y consumiendo ciclos de CPU bajo el control del sistema operativo.

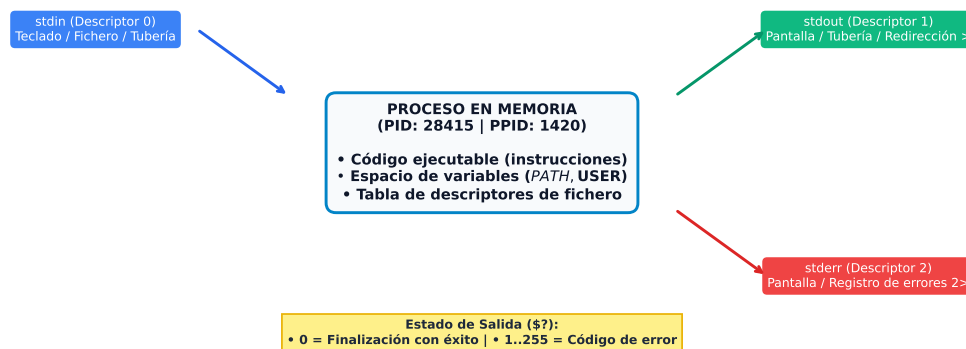


Figure 1.1: Modelo de proceso en memoria y descriptores estándar de entrada/salida (**stdin**, **stdout**, **stderr**) bajo el sistema operativo. Figura original adaptada de las presentaciones docentes de la asignatura.

1.3.1 El ciclo de vida de un proceso

Un proceso no es un bloque inerte. Desde que nace hasta que termina, transita por diversos estados de ejecución:

Un proceso Unix transita durante su vida por estados de ejecución bien definidos —en ejecución, en espera, detenido, y en casos patológicos "zombie"—; no es una entidad estática mientras dura.

- **En ejecución (*running*):** El proceso está ejecutando activamente instrucciones en la CPU o esperando su turno inmediato en la cola del planificador.
- **En espera (*waiting* / *sleeping*):** El proceso aguarda que ocurra un evento externo, por ejemplo que el disco termine de leer un archivo de secuencias o que llegue un paquete de red.
- **Detenido (*stopped*):** El proceso ha sido pausado temporalmente (por ejemplo, mediante una señal interactiva del usuario para depuración).
- **Zombie:** El proceso ha finalizado su tarea pero su proceso padre aún no ha leído su código de terminación. Permanece en la tabla de procesos como un registro residual que no consume CPU ni memoria.

1.3.2 Jerarquía y aislamiento de memoria

En Linux, todos los procesos forman una estructura estrictamente jerárquica con forma de árbol cuya raíz es el proceso primordial del sistema (**init** o **systemd**, con identificador PID 1).

Los procesos de un sistema Linux se organizan en un árbol con `init` como raíz; cada proceso hijo obtenido con `fork()` recibe un PID distinto del de su padre pero hereda sus valores iniciales.

Para crear un nuevo proceso, el sistema operativo utiliza la llamada al sistema `fork()`, que duplica el proceso actual generando un hijo con su propio identificador numérico (*Process ID* o PID).

Cada proceso Unix tiene un espacio de memoria aislado del de los demás; la comunicación entre procesos requiere un mecanismo explícito (señales o memoria compartida gestionada por el núcleo), no ocurre por defecto.

Este aislamiento garantiza la estabilidad de los servidores científicos: si un alineador de secuencias comete un error de segmentación o agota la memoria, solo muere ese proceso individual, sin corromper la memoria de la Shell ni tumbar los análisis de otros investigadores en el mismo servidor.

1.3.3 Privilegios: usuarios y superusuario root

El superusuario de un sistema Linux se llama `root` y tiene UID 0; un usuario sin privilegios puede escalar temporalmente su nivel de privilegio solo si pertenece a un grupo autorizado para ello, y debe hacerlo de forma explícita, no por defecto.

En un entorno de cálculo compartido, los usuarios ordinarios tienen permiso para crear, modificar y borrar exclusivamente sus propios archivos de trabajo. Solo el administrador (`root`) o usuarios autorizados mediante la orden `sudo` pueden alterar archivos del sistema operativo o instalar controladores globales de hardware.

1.3.4 El estado de salida de un proceso

Cuando un proceso finaliza, devuelve al sistema operativo un número entero comprendido entre 0 y 255 conocido como **código o estado de salida** (*exit status*). En Bash, este valor queda inmediatamente registrado en la variable especial `$?`.

El estado de salida de un proceso en Bash queda disponible en `$?` inmediatamente después de su ejecución, y vale 0 por convención cuando el proceso no detectó error; no certifica que el resultado sea correcto, solo que el proceso terminó de cierta manera.

Un estado de salida 0 indica que el programa se ejecutó sin detectar errores internos; cualquier valor distinto de 0 (por ejemplo 1, 2 o 127) indica un tipo específico de fallo. Esta convención es la base que permite a los scripts y flujos automatizados de bioinformática decidir si continúan con el siguiente paso del análisis o se detienen ante un error. Se reproduce con `verification/chapter_results.sh exit_status`.

Resolución de la predicción 3. El estado de salida 0 (`$? = 0`) solo certifica que el proceso finalizó conforme a las convenciones del sistema operativo sin abortar por error técnico. No valida la veracidad, calidad ni adecuación del modelo biológico; la comprobación científica requiere siempre la inspección explícita de los resultados generados.

1.4 El sistema de archivos: el mapa de navegación

A diferencia de otros sistemas operativos que asignan letras independientes a cada disco o partición (C:, D:), Linux organiza todo su almacenamiento en un **único árbol jerárquico universal** que parte de una raíz común representada por una barra diagonal: `/`.

Todo lo que existe en el sistema —discos locales, discos de red compartidos, memorias USB, archivos de configuración y datos de pacientes— se encuentra ubicado en alguna rama de este único árbol.

1.4.1 Directorio de trabajo: el suelo donde pisamos

Cada sesión de Shell y cada proceso tiene en todo momento un **directorio de trabajo** (*working directory* o *cwd*): es la posición actual dentro del árbol desde la cual se interpretan todas las órdenes relativas.

- **Ruta absoluta:** Es la dirección completa e inequívoca de un fichero desde la raíz del sistema. Comienza siempre por /, por ejemplo `/home/alumno/TC01/datos/secuencias.fasta`. Es válida sin importar en qué carpeta se encuentre usted.
- **Ruta relativa:** Es una dirección calculada a partir del directorio donde se encuentra la Shell en ese instante. No empieza por /, por ejemplo `datos/secuencias.fasta`.

Una ruta relativa se interpreta respecto al directorio de trabajo del proceso que la evalúa; la misma cadena de texto puede referirse a ficheros distintos según ese directorio.

Resolución de la predicción 1. La expresión `datos/muestra.fasta` es una ruta relativa: su significado exacto cambia según cuál sea el directorio de trabajo actual (`$PWD`) del proceso en cada momento. Solo una ruta absoluta (que comienza por /) identifica unívocamente la misma ubicación en el árbol de directorios.

La gran mayoría de los errores en las primeras semanas de trabajo en bioinformática (el temido mensaje `No such file or directory`) ocurren porque el estudiante asume que está en una carpeta distinta de la real e intenta usar una ruta relativa que no coincide con su posición en el árbol.

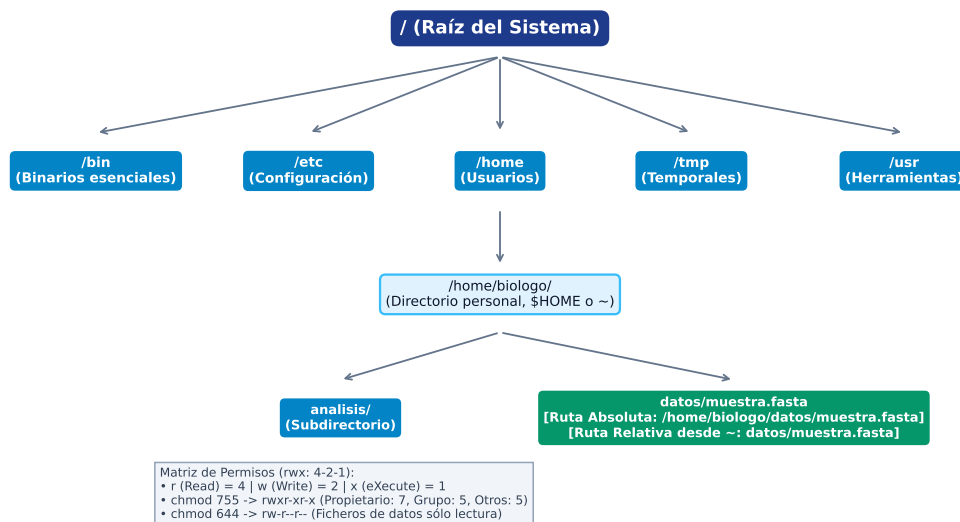


Figure 1.2: Jerarquía en árbol de raíz única (/), resolución de rutas absolutas frente a relativas y matriz octal de permisos (rwx). Figura original adaptada de las presentaciones docentes de la asignatura.

Para el árbol de ejemplo, la ruta relativa `datos/tc01_mini.fasta` solo resuelve al fichero real cuando el directorio de trabajo es `/home/alumno/TC01`; desde `/home/alumno` señala a una ruta inexistente.

	/	
	home/	
	alumno/	
	TC01/	
	datos/tc01_mini.fasta	
Directorio de trabajo	datos/tc01_mini.fasta	señala a
/home/alumno/TC01	/home/alumno/TC01/datos/tc01_mini.fasta	(existe)
/home/alumno	/home/alumno/datos/tc01_mini.fasta	(no existe)

Figure 1.3: La misma ruta relativa apunta a ficheros distintos —o a ninguno— según el directorio de trabajo. Figura original. Texto alternativo: un árbol de directorios muestra la ruta hasta el fichero; una tabla muestra que la ruta relativa `datos/tc01_mini.fasta` solo resuelve correctamente cuando el directorio de trabajo es TC01.

1.4.2 La brújula de la terminal

Para moverse con agilidad y escribir rutas sin redundancia, la Shell dispone de referencias estándar:

- **.** (**punto**): Representa el directorio actual.
- **..** (**dos puntos**): Representa el directorio padre (un nivel inmediatamente superior en el árbol).
- **~** (**tilde / virgulilla**): Representa el directorio personal del usuario (*home directory*, habitualmente `/home/nombre_usuario`).
- **pwd** (**print working directory**): Imprime en pantalla la ruta absoluta exacta del directorio donde se encuentra [1, pp. 51–54].

Tras `mkdir -p a/b/c && cd a/b/c`, la ruta relativa `../..` conduce al directorio `a`; `pwd` tras ambas operaciones lo confirma. Se reproduce en un entorno limpio con `verification/chapter_results.sh relative_path`.

1.5 Navegación e inspección sin alterar

El trabajo cotidiano en la línea de comandos sigue una regla de oro: **inspeccionar antes de modificar**. Modificar o borrar a ciegas es la causa principal de pérdidas irreparables de datos experimentales.

Orden	Efecto	Nota de control
<code>pwd</code>	Imprime el directorio de trabajo absoluto	La primera orden a ejecutar ante cualquier duda de ubicación.
<code>ls</code>	Lista los ficheros del directorio indicado o actual	<code>ls -l</code> añade permisos, propietario y tamaño; <code>ls -a</code> muestra ocultos; <code>ls -lh</code> muestra tamaños legibles (KB, MB, GB).
<code>cd RUTA</code>	Cambia el directorio de trabajo a la ruta dada	<code>cd</code> solo vuelve al home; <code>cd -</code> regresa al directorio anterior; <code>cd ..</code> sube un nivel.

Orden	Efecto	Nota de control
<code>mkdir -p RUTA</code>	Crea directorios intermedios necesarios	La opción <code>-p</code> evita errores si los padres no existen o si la carpeta ya existía.
<code>cp ORIGEN DESTINO</code>	Copia ficheros o carpetas (<code>cp -r</code>)	Conserva el original intacto; confirme el destino antes de sobrescribir.
<code>mv ORIGEN DESTINO</code>	Mueve o renombra un fichero o carpeta	No deja copia en el origen; la operación de mover es instantánea en el mismo disco.
<code>touch FICHERO</code>	Crea un fichero vacío o actualiza su fecha	No altera el contenido interno si el fichero ya existía.
<code>file FICHERO</code>	Determina la naturaleza real del fichero	Analiza los bytes de cabecera (<i>magic numbers</i>), no la extensión.
<code>head / tail</code>	Muestra las primeras / últimas líneas	Por defecto 10 líneas; use <code>-n NUM</code> para especificar la cantidad.
<code>less FICHERO</code>	Visor interactivo paginado	Permite explorar ficheros de gigabytes sin cargarlos enteros en la memoria RAM.
<code>wc FICHERO</code>	Cuenta líneas (<code>-l</code>), palabras (<code>-w</code>) y bytes (<code>-c</code>)	En biología, recuerde que <code>wc -l</code> cuenta saltos de línea, no necesariamente entidades biológicas.

`pwd`, `ls`, `cd`, `mkdir -p`, `cp`, `mv`, `touch` y las órdenes de inspección (`file`, `head`, `tail`, `less`, `wc`) son las órdenes mínimas de navegación, organización e inspección documentadas en GNU Coreutils.

1.5.1 Inspección segura de datos biológicos masivos

Uno de los errores más destructivos al comenzar en bioinformática es hacer doble clic sobre un archivo de secuenciación FASTQ de 15 GB para "abrirlo" con un editor de texto gráfico convencional (como el Bloc de notas o Word). El programa intentará cargar los 15 gigabytes íntegros en la memoria RAM, congelando el sistema operativo.

Las utilidades de Unix están diseñadas para manejar flujos de datos sin importar su tamaño:

- `head -n 20 archivo.fastq`: Lee únicamente las primeras 20 líneas directamente del flujo del disco y se detiene al instante, consumiendo unos pocos kilobytes de memoria.
- `tail -n 20 archivo.fastq`: Muestra las últimas 20 líneas (útil para comprobar si una descarga o simulación finalizó correctamente).
- `less archivo.fastq`: Abre un visor interactivo que lee el archivo por bloques a medida que usted avanza por la pantalla. Puede desplazarse con la **barra espaciadora** (página adelante), con la tecla **b** (página atrás), buscar palabras escribiendo `/patrón` y pulsando Enter (navegando entre coincidencias con **n** y **N**), y salir de forma limpia y segura pulsando la tecla **q**.

1.5.2 La trampa de contar líneas frente a contar secuencias

`file` identifica el tipo de un fichero a partir de su contenido, no de su extensión; `data/tc01_sample.fasta` tiene 19 líneas, verificable con `wc -l` y a mano.

La orden `wc -l` cuenta estrictamente caracteres de fin de línea (`\n`). En un fichero FASTA donde cada secuencia puede estar dividida en múltiples líneas de nucleótidos o aminoácidos, `wc -l` devolverá el número total de líneas del archivo, no el número de secuencias biológicas. Para contar secuencias reales es necesario filtrar las cabeceras (como aprenderá en TC-CH02 con `grep -c '^>'`). Se reproduce con `verification/chapter_results.sh file_type` y `line_count_sample`.

1.5.3 El problema silencioso de los saltos de línea Windows (CRLF)

Un problema frecuente ocurre cuando un fichero de texto se descarga de una hoja de cálculo Excel en Windows o se edita en un entorno Windows y luego se procesa en un servidor Linux. Mientras que Linux y Unix representan el fin de línea con un único carácter *Line Feed* (`\n` o `LF`), Windows utiliza una combinación de dos caracteres: *Carriage Return* seguido de *Line Feed* (`\r\n` o `CRLF`).

El comando `file` analiza los primeros bytes del archivo y detecta de inmediato esta discrepancia:

`file` detecta caracteres de fin de línea DOS/Windows (CRLF) distinguiéndolos de los terminadores nativos de Unix (LF), lo que evita fallos sutiles en scripts al importar datos. Se reproduce con `verification/chapter_results.sh crlf_detection`.

1.5.4 Ficheros ocultos y configuración

En Linux, cualquier archivo o carpeta cuyo nombre comience por un punto (como `.bashrc`, `.gitignore` o la carpeta de control de versiones `.git`) se considera un **fichero oculto**. Estos ficheros contienen habitualmente configuraciones del sistema o metadatos y la orden `ls` los omite por defecto para mantener limpia la vista del directorio.

Los ficheros y directorios ocultos que comienzan por un punto se omiten en la salida estándar de `ls` y se listan explícitamente usando `ls -a` o `ls -la`. Se reproduce con `verification/chapter_results.sh hidden_files_listing`.

1.5.5 La verdad de su sistema: `-help` y `man`

Nunca confíe ciegamente en ejemplos de código copiados de foros de internet sin contrastar las opciones de la herramienta en su propia instalación. Diversas distribuciones de Linux y versiones de herramientas bioinformáticas tienen parámetros ligeramente distintos.

`-help` y `man` ORDEN documentan la versión de la orden realmente instalada; un ejemplo copiado de otra fuente puede corresponder a otra versión o a otra utilidad de nombre igual.

1.6 Enlaces duros y enlaces simbólicos (`ln` y `ln -s`)

En el sistema de archivos de Linux, la información de un fichero se divide en dos partes: el bloque de datos físicos almacenados en el disco y una estructura de metadatos del sistema operativo denominada **inodo** (*inode*). El inodo almacena el tamaño del fichero, sus permisos, sus fechas de modificación y los punteros a los sectores del disco donde residen sus datos, pero **no** almacena su nombre. El nombre es simplemente una entrada en una tabla de directorio que apunta a un número de inodo.

Esta arquitectura permite crear dos tipos fundamentales de enlaces:

- **Enlace duro (*hard link*):** Creado con `ln origen enlace`. Crea una nueva entrada de directorio con un nombre nuevo que apunta al **mismo número de inodo** que el fichero original. Ambos nombres tienen la misma categoría; si se borra uno de ellos, los datos físicos en el disco permanecen intactos mientras quede al menos un enlace duro apuntando al inodo (el contador de enlaces `st_nlink` sea mayor que 0). No pueden crearse entre sistemas de archivos distintos ni sobre directorios.
- **Enlace simbólico (*symbolic link* o *symlink*):** Creado con `ln -s origen enlace`. Es un fichero especial independiente (con su propio inodo) cuyo contenido interno es simplemente una cadena de texto que contiene la ruta hacia el fichero o directorio de destino. Si se borra el enlace simbólico, el fichero original no se ve afectado; si se borra o mueve el fichero original, el enlace simbólico queda "roto" (*dangling symlink*).

Un enlace simbólico `ln -s` crea un puntero de ruta independiente cuya eliminación deja el fichero original intacto, mientras que un enlace duro `ln` comparte el mismo inodo exacto y el mismo contador de enlaces. Se reproduce con `verification/chapter_results.sh symlink_creates_pointer`.

1.6.1 El valor de los enlaces simbólicos en bioinformática

En proyectos de genómica y transcriptómica es habitual disponer de un genoma de referencia de la especie humana (como `GRCh38.fa`, que supera los 3 GB de tamaño) o colecciones de archivos FASTQ de secuenciación masiva almacenados en un disco central del laboratorio.

En bioinformática, los enlaces simbólicos permiten que múltiples proyectos independientes referencien conjuntos de datos masivos compartidos (como genomas de referencia o lecturas de secuenciación) sin duplicar almacenamiento físico en el disco.

En lugar de copiar 3 GB en cada carpeta de análisis de cada estudiante (lo que saturaría el servidor en cuestión de días), se crea un enlace simbólico `ln -s /datos/referencias/GRCh38.fa ./genoma_ref.fa`. El software procesa el enlace exactamente igual que el archivo original, pero el consumo de almacenamiento adicional es de apenas unos pocos bytes.

1.7 Propiedad y permisos

La seguridad y la colaboración en un entorno multiusuario se basan en el modelo de permisos de Unix. Cada fichero y carpeta tiene asignado un **usuario propietario** (*user*, `u`) y un **grupo propietario** (*group*, `g`), y define permisos independientes para tres categorías:

1. **Propietario (u):** El usuario dueño del fichero.
2. **Grupo (g):** Los usuarios que pertenecen al grupo de trabajo o proyecto.
3. **Otros (o):** Todos los demás usuarios del sistema.

Para cada categoría existen tres permisos elementales: lectura (`r`), escritura (`w`) y ejecución (`x`). La orden `ls -l` muestra estos permisos en una cadena de 10 caracteres (por ejemplo `-rwxr-xr-`): el primer carácter indica si es un fichero regular (`-`), un directorio (`d`) o un enlace (`l`), seguido de los nueve bits de permisos agrupados de tres en tres (`rw` para usuario, `r` para grupo, `r` para otros).

Cada fichero Unix tiene un permiso de lectura, escritura y ejecución independiente para su propietario, su grupo y el resto de usuarios; `ls -l` muestra los nueve bits resultantes en ese orden.

1.7.1 El significado especial de la ejecución (x) en directorios

En un fichero ordinario de texto o script, el bit **x** otorga permiso para ejecutarlo como un programa. Sin embargo, en un **directorio**, el bit **x** significa **permiso de paso o acceso**: otorga la capacidad de entrar en la carpeta con `cd`, atravesarla para alcanzar subcarpetas o acceder a los inodos de los ficheros contenidos en ella. Si un directorio tiene permisos de lectura (**r**) pero carece de ejecución (**x**), usted podrá ver la lista de nombres de los ficheros, pero no podrá leer su contenido ni sus tamaños.

1.7.2 Modificación de permisos con `chmod`

La orden `chmod` (*change mode*) permite modificar los permisos mediante dos nomenclaturas:

Forma	Ejemplo	Efecto
Simbólica	<code>chmod u+x script.sh</code>	Añade permiso de ejecución exclusivamente al propietario.
Simbólica	<code>chmod g-w,o-r datos/</code>	Retira escritura al grupo y lectura al resto de usuarios.
Octal / Numérica	<code>chmod 751 script.sh</code>	Asigna r =4, w =2, x =1 sumados por categoría: usuario 7 (rw x), grupo 5 (r - x), otros 1 (- x).
Octal / Numérica	<code>chmod 644 tabla.tsv</code>	Permisos estándar de datos: usuario 6 (rw -), grupo 4 (r -), otros 4 (r -).
Octal / Numérica	<code>chmod 755 proyecto/</code>	Permisos estándar de carpetas y programas públicos: usuario 7 (rw x), grupo 5 (r - x), otros 5 (r - x).
Propiedad	<code>chown usuario:grupo fichero</code>	Modifica el propietario y el grupo asignados al fichero.

`chmod` admite forma simbólica (**u**+**x**, **g**-**w**, ...) y forma numérica (suma de **r**=4, **w**=2, **x**=1 por categoría, p. ej. 751); `chown` cambia propietario y grupo. Se reproduce con `verification/chapter_results.sh chmod_numeric_matches_symbolic`.

Listing 1.1: Algoritmo formal en pseudocódigo para el diagnóstico en dos fases de acceso a rutas y permisos.

```

ALGORITMO: Verificacion_Acceso_Rutas_Permisos
ENTRADA:
  - Ruta_Objeto: Cadena de texto con la ruta relativa o absoluta
  - Modo_Requerido: Conjunto de permisos requeridos {Lectura (R), Escritura (W),
    Ejecucion (X)}
  - Directorio_Actual: Directorio de trabajo del proceso (PWD)
SALIDA:
  - Estado_Acceso: {ACCESO_PERMITIDO, ERROR_NO_EXISTE (ENOENT),
    ERROR_PERMISO_DENEGADO (EACCES)}
COMPLEJIDAD: Tiempo O(D) donde D es la profundidad del arbol de directorios, Espacio
O(1)

1. FASE 1 - RESOLUCION DE RUTA Y COMPROBACION DE EXISTENCIA:
  Si Ruta_Objeto NO comienza por '/' entonces:
    Ruta_Canonica <- Concatenar(Directorio_Actual, '/', Ruta_Objeto)
  Sino:
    Ruta_Canonica <- Ruta_Objeto

```

```

Si Ruta_Canonica NO existe en el sistema de archivos entonces:
    Retornar ERROR_NO_EXISTE (Diagnostico: Comprobar PWD o nombre de archivo)

2. FASE 2 - EVALUACION DE ATRIBUTOS Y MATRIZ DE PERMISOS:
Para cada directorio ancestro en Ruta_Canonica:
    Si el usuario NO tiene permiso de ejecucion (x) en el ancestro entonces:
        Retornar ERROR_PERMISO_DENEGADO (Fallo de traversal en directorio padre)

Obtener mascara de permisos (rwx) del nodo destino para el usuario actual:
Si Modo_Requerido contiene 'Lectura' y NO_TIENE(r) entonces:
    Retornar ERROR_PERMISO_DENEGADO (Faltan permisos de lectura)
Si Modo_Requerido contiene 'Escritura' y NO_TIENE(w) entonces:
    Retornar ERROR_PERMISO_DENEGADO (Faltan permisos de escritura o sistema de
        solo lectura)
Si Modo_Requerido contiene 'Ejecucion' y NO_TIENE(x) entonces:
    Retornar ERROR_PERMISO_DENEGADO (El archivo no tiene flag ejecutable activado)

3. TERMINACION:
    Retornar ACCESO_PERMITIDO

```

Resolución de la predicción 2. Tener permiso de lectura (**r**) permite examinar los bytes de un script o binario, pero no ejecutarlo como programa independiente si carece del bit de ejecución (**x**). En Unix, los tres permisos (**r**, **w**, **x**) son ortogonales e independientes.

1.8 Ejecutar un archivo y variables de entorno

Para que un archivo pueda ejecutarse como un programa en la terminal, deben cumplirse simultáneamente dos condiciones obligatorias:

1. **Localización:** La Shell debe saber exactamente dónde encontrar el fichero en el disco.
2. **Permiso de ejecución:** El fichero debe tener activo el bit **x** para el usuario que lo invoca.

Ejecutar un archivo exige que la Shell pueda localizarlo (por `$PATH` o ruta explícita) y que tenga permiso de ejecución para el usuario que lo invoca; ninguna de las dos condiciones es opcional.

Si intenta ejecutar un fichero que carece del bit **x**, Bash responderá con el error `Permission denied`.

1.8.1 La variable `$PATH` frente a la ruta explícita `./`

Cuando usted escribe el nombre de un comando en la terminal sin barras (por ejemplo `ls` o `fastqc`), Bash no inspecciona mágicamente todo el disco duro. En su lugar, consulta la variable de entorno `$PATH`, que contiene una lista ordenada de directorios separados por dos puntos (:), por ejemplo `/usr/local/bin:/usr/bin:/bin`. Bash recorre esos directorios en orden estricto de izquierda a derecha y ejecuta el primer fichero coincidente que encuentre.

Por motivos de seguridad, el directorio actual (`.`) **no** suele estar incluido en `$PATH`. Si escribe un script llamado `analisis.sh` en su carpeta actual y teclea simplemente `analisis.sh`, Bash dirá `command not found`. Para forzar a Bash a ejecutar el archivo de la carpeta actual sin buscar en `$PATH`, debe proporcionar una ruta explícita anteponiendo `./`, escribiendo: `./analisis.sh`.

Cuando se invoca un ejecutable por nombre, Bash lo busca en los directorios listados en `$PATH`, en orden; una ruta explícita omite esa búsqueda. Se reproduce con `verification/chapter_results.sh path_resolution`.

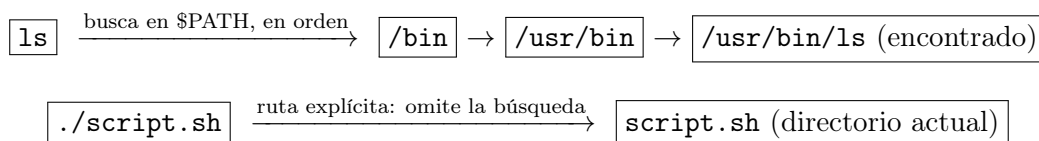


Figure 1.4: Arriba, el nombre `ls` se busca en cada directorio de `$PATH` hasta encontrar `/usr/bin/ls`; abajo, la ruta explícita `./script.sh` evita esa búsqueda y usa el fichero del directorio actual. Figura original. Texto alternativo: arriba el nombre `ls` se busca en cada directorio de `PATH` hasta encontrar `/usr/bin/ls`; abajo la ruta explícita `./script.sh` evita esa búsqueda y usa el fichero del directorio actual.

El diagrama ilustra que localizar un ejecutable por nombre recorre `$PATH` en orden hasta encontrarlo, mientras que una ruta explícita como `./script.sh` omite esa búsqueda y usa directamente el fichero indicado.

1.8.2 El Shebang: el contrato de ejecución

La primera línea de un script ejecutable en Unix debe contener la directiva conocida como **shebang** (`#!`):

```
#!/usr/bin/env bash
```

Cuando el núcleo del sistema operativo intenta ejecutar un archivo de texto con permiso `x`, lee los dos primeros bytes (`#!`). Al detectarlos, el kernel sabe que no se trata de código máquina binario, sino de un script de texto, y utiliza la ruta que sigue inmediatamente después para invocar el intérprete correspondiente (en este caso, Bash) y pasarle el archivo como argumento.

1.8.3 Variables locales frente a variables de entorno exportadas

En Bash podemos crear variables asignando un valor sin espacios alrededor del signo igual (por ejemplo, `MUESTRA="paciente_01"`). Sin embargo, existe una diferencia crucial de visibilidad:

- **Variable local de Shell:** Solo existe dentro de la sesión interactiva donde se creó. Si desde esa sesión se ejecuta un script o un comando externo, el subproceso hijo no verá la variable.
- **Variable de entorno exportada:** Se declara con `export VARIABLE=valor`. El sistema operativo la copia en el bloque de entorno de todos los procesos hijos que se lancen a partir de ese momento.

Una variable de Bash ordinaria (`NOMBRE=valor`) es local a la sesión que la crea; solo se convierte en variable de entorno disponible para los procesos que esa sesión lance si se exporta con `export NOMBRE` o `export NOMBRE=valor`. Se reproduce con `verification/chapter_results.sh export_makes_visible_to_subprocess`.

Resolución de la predicción 4. Una variable asignada localmente en Bash (`VAR=valor`) solo existe en la memoria del intérprete actual; los subprocesos hijos no la reciben a menos que se exporte explícitamente al entorno con `export VAR`.

1.8.4 Variables predefinidas y configuración permanente

El sistema operativo y la Shell definen de forma automática una serie de variables fundamentales:

`$HOME`, `$PWD`, `$OLDPWD` y `$PATH` son variables de entorno ya definidas por el sistema: el directorio personal, el directorio de trabajo actual, el directorio de trabajo anterior tras el último `cd`, y la lista de búsqueda de ejecutables, respectivamente. Se reproduce con `verification/chapter_results.sh oldpwd_tracks_previous_cd`.

Si desea que una variable esté siempre disponible en cada terminal que abra (por ejemplo, añadir una carpeta personal de herramientas con `export PATH="$HOME/bin:$PATH"`), debe añadir esa línea al archivo de configuración de su cuenta: `~/.bashrc`. Para recargar los cambios en la terminal actual sin tener que cerrarla y volverla a abrir, se utiliza la orden `source ~/.bashrc`.

1.9 Ergonomía y productividad en la terminal

Trabajar en la línea de comandos con torpeza tecleando letra por letra cada ruta kilométrica genera fatiga y errores tipográficos. Bash incorpora la biblioteca GNU Readline, que ofrece atajos de teclado fundamentales:

Los atajos de teclado de GNU Readline en Bash, como `Tab` para autocompletar, `Ctrl+C` para interrumpir, `Ctrl+L` para limpiar y `Ctrl+A`/`Ctrl+E` para saltar a los extremos de la línea, constituyen la ergonomía interactiva fundamental en bioinformática.

- **Tab (Autocompletado):** La tecla más valiosa de un bioinformático. Si escribe `cd dat` y pulsa `Tab`, Bash autocompletará `datos/`. Si hay varias opciones, pulsar `Tab` dos veces seguidas mostrará todas las alternativas. Utilizar siempre el autocompletado garantiza que no se cometan erratas en nombres largos de muestras.
- **Ctrl+C:** Envía la señal `SIGINT` al proceso en primer plano, abortando de inmediato cualquier comando o bucle infinito que se haya descontrolado.
- **Ctrl+L:** Limpia la pantalla de la terminal dejando el cursor arriba, equivalente visual de la orden `clear`.
- **Ctrl+A y Ctrl+E:** Mueven el cursor instantáneamente al principio (`Ctrl+A`) o al final (`Ctrl+E`) de la línea de comandos sin necesidad de mantener pulsadas las flechas del teclado.
- **Ctrl+U y Ctrl+K:** Borran desde el cursor hasta el principio (`Ctrl+U`) o hasta el final (`Ctrl+K`) de la línea.
- **Alt+. (o Esc seguido de punto):** Inserta de forma inmediata el último argumento del comando anterior (extremadamente útil para navegar a una carpeta recién creada con `mkdir`).

1.9.1 El historial y sus límites

Bash registra las órdenes ejecutadas en un historial en memoria que se guarda en el archivo `~/.bash_history`. Pulsando `Ctrl+R` puede realizar una búsqueda inversa interactiva en su historial tecleando fragmentos de órdenes anteriores.

`history` refleja únicamente la sesión interactiva donde se ejecuta, tiene un límite de líneas conservadas, y puede omitir órdenes ejecutadas en otra terminal; no es, por sí solo, un procedimiento documentado.

Un volcado del historial no sustituye un protocolo reproducible: en el historial se mezclan erratas, intentos fallidos y órdenes irrelevantes. Un procedimiento formal debe quedar plasmado en un script limpio y un documento explicativo.

1.10 Espacio de almacenamiento y organización del proyecto

En bioinformática es habitual procesar proyectos que generan terabytes de archivos temporales. Monitorear el estado de los discos evita fallos críticos durante ejecuciones que duran días:

- **df -h (*disk free*):** Muestra el espacio total, usado y disponible en todos los sistemas de archivos montados en el equipo, en formato legible por humanos (-h, megabytes o gigabytes).
- **du -sh RUTA (*disk usage*):** Calcula el espacio real ocupado por un directorio específico y todo su contenido de forma resumida (-s).

df inspecciona el espacio de disco libre y ocupado en los sistemas de archivos montados, mientras que **du** calcula el espacio consumido por ficheros y directorios específicos. Se reproduce con `verification/chapter_results.sh disk_usage_check`.

1.10.1 Estructura canónica de un proyecto bioinformático

Siguiendo las recomendaciones internacionales de buenas prácticas [1], todo proyecto de análisis computacional debe mantener una estructura de carpetas estricta y predecible:

- **data/raw/:** Datos originales crudos (lecturas de secuenciador, microarrays). **Solo lectura:** jamás se modifican.
- **data/processed/:** Datos limpios, filtrados y transformados.
- **scripts/:** Código fuente y scripts de automatización del análisis.
- **results/:** Gráficos, tablas finales y reportes de salida.
- **README.md:** Documento que explica el objetivo del proyecto, las dependencias y cómo ejecutar el análisis.

1.11 Antes de una operación que sobrescribe o elimina

En la línea de comandos de Unix **no existe papelera de reciclaje**. Cuando se ejecuta una orden de eliminación como `rm fichero`, el sistema operativo desvincula el inodo y marca los bloques del disco como disponibles para ser sobrescritos de inmediato. El borrado es instantáneo, silencioso e irreversible.

Para evitar catástrofes accidentales, aplique siempre el **protocolo preventivo de dos pasos**:

1. **Verificar el directorio actual con `pwd`:** Asegúrese de que está en la carpeta de trabajo correcta y no en el directorio raíz o en su carpeta personal.
2. **Inspeccionar el objetivo exacto con `ls -ld - RUTA`:** Compruebe que la ruta señala con absoluta exactitud al fichero o directorio que desea eliminar antes de sustituir `ls -ld` por la orden destructiva.

Antes de cualquier operación que pueda sobrescribir o eliminar datos conviene: confirmar el directorio con `pwd` e inspeccionar el objetivo exacto con `ls -ld - RUTA` antes de ejecutar nada; solo se actúa sobre objetivos explícitos ya comprobados de esta forma.

El caso más destructivo y peligroso en la historia de la administración de sistemas ocurre al emplear variables no definidas dentro de órdenes recursivas (por ejemplo, `rm -rf "$CARPETA_TEMPORAL"/*`, que si la variable está vacía se expande accidentalmente a `rm -rf "/*"`, borrando el sistema entero). Este problema se debe al mecanismo de expansión de variables de Bash, que estudiaremos en detalle en el capítulo TC-CH02 (§TC02-SEC-SAFEVAR).

1.12 Reproducibilidad, empaquetado e integridad

La reproducibilidad de un análisis computacional exige documentar con precisión matemática el contexto en el que se obtuvieron los resultados. Un registro reproducible debe recoger de forma explícita siete campos esenciales:

Un registro mínimo de entorno reproducible incluye, como mínimo: sistema operativo, versión de Bash, origen y versión de los datos, fecha, las órdenes ejecutadas, el resultado esperado y las limitaciones conocidas.

Delimitación conceptual: reproducibilidad y replicabilidad. En este capítulo y a lo largo de este libro, **reproducibilidad computacional** significa la capacidad de reejecutar exactamente el mismo procedimiento analítico utilizando los mismos datos brutos y el mismo entorno de software registrado, obteniendo bit a bit el mismo resultado numérico.

Por el contrario, obtener conclusiones convergentes o equivalentes utilizando una cohorte independiente, nuevos datos biológicos o una implementación algorítmica distinta responde a una dimensión científica complementaria, habitualmente denominada **replicabilidad** o validez externa.

- *Escenario A (Reproducibilidad):* Una investigadora ejecuta el script publicado sobre el archivo de datos original y verifica que el hash SHA-256 de la tabla de salida coincide de forma unívoca con el del artículo.
- *Escenario B (Replicabilidad):* Otro laboratorio recoge una nueva muestra de pacientes, aplica un protocolo análogo y confirma que el biomarcador mantiene significación estadística.

`verification/check_environment_registration.sh` genera y comprueba un registro con estos siete campos a partir de `uname -a`, `bash -version`, `date` y el manifiesto de `data/` [1, pp. 67–70].

1.12.1 Empaquetado portable con tar

Para compartir un proyecto con colaboradores o subirlo a un repositorio público preservando su estructura interna de directorios y permisos, se utiliza **tar** (*tape archive*) combinado con compresión **gzip**:

- **Crear paquete:** `tar -czf entrega.tar.gz carpeta/` (opciones: **c** = crear, **z** = comprimir con gzip, **f** = nombre del archivo resultante).
- **Inspeccionar sin extraer:** `tar -tzf entrega.tar.gz` (opción **t** = listar contenido; permite verificar la estructura antes de volcar archivos en el disco).
- **Extraer contenido:** `tar -xzf entrega.tar.gz` (opción **x** = extraer).

Empaquetar el directorio `data/` con `tar -czf` y volver a extraerlo en un directorio distinto reproduce el mismo número de ficheros y las mismas sumas `sha256sum` que el original. Se reproduce con `verification/check_package_roundtrip.sh`.

1.12.2 Integridad criptográfica con `sha256sum`

Un archivo puede corromperse durante una descarga por red o por un fallo en el soporte físico. Las funciones de dispersión criptográfica (*hash*) como SHA-256 calculan una huella digital única de 64 caracteres hexadecimales a partir del contenido exacto de los bytes del archivo. Si cambia un solo bit en un archivo de 50 GB, el hash resultante cambiará por completo.

`sha256sum -c` detecta cualquier cambio de al menos un byte respecto al manifiesto registrado, pero no distingue un cambio intencionado de una corrupción accidental ni valida el significado del contenido. Se reproduce con `verification/check_manifest_integrity.sh`.

1.13 Método de verificación y errores frecuentes

Para adquirir maestría y autonomía técnica, aplique de forma sistemática el ciclo metodológico del curso ante cualquier instrucción propuesta por usted o por un asistente de inteligencia artificial:

El ciclo predecir, probar en pequeño, inspeccionar cada etapa, contrastar con la estructura esperada y registrar reduce la aceptación de una salida plausible pero incorrecta; este capítulo lo introduce y TC-CH02 lo recupera sin volver a enseñarlo.

1. **Predecir:** Formule en su mente (o en voz alta) qué espera exactamente que haga el comando antes de presionar la tecla Enter.
2. **Probar en pequeño:** Ejecute primero la orden sobre un subconjunto diminuto de datos (un archivo de 5 líneas) donde conozca el resultado exacto de antemano.
3. **Inspeccionar cada etapa:** Compruebe el estado de salida (`echo $?`) y revise la salida intermedia.
4. **Contrastar con la estructura:** Verifique si el resultado tiene sentido biológico y dimensional (si esperaba 4 muestras, un resultado con 0 o 400 es señal de alarma).
5. **Registrar:** Guarde el comando validado en su script documentado.

Síntoma observable	Causa probable	Comprobación y corrección
No such file or directory	Directorio de trabajo incorrecto o errata en la ruta relativa.	Ejecutar <code>pwd</code> ; usar la tecla <code>Tab</code> para autocompletar la ruta real.
command not found	Orden no instalada o script en la carpeta actual invocado sin <code>./</code> .	Verificar con <code>command -v ORDEN</code> ; anteponer <code>./</code> si es un script local.
Permission denied	Falta el bit de ejecución (<code>x</code>) o de lectura (<code>r</code>).	Comprobar con <code>ls -l</code> ; asignar <code>chmod u+x</code> solo sobre scripts propios revisados.
Un fichero con espacios "desaparece" o divide el comando en dos	El espacio separa argumentos en Bash.	Entrecomillar siempre las rutas con espacios (" <code>fichero con espacios.txt</code> ").

Síntoma observable	Causa probable	Comprobación y corrección
Una variable no llega a un script ejecutado	La variable fue definida como local y no fue exportada.	Ejecutar <code>export NOMBRE</code> antes de lanzar el subproceso o definirla en la misma línea.

Los cinco síntomas de esta tabla corresponden a los errores de principiante más frecuentes documentados en la auditoría del módulo legacy y en la presentación oficial de Fundamentos de Linux.

Compruebe antes de corregir · Diagnóstico en dos fases. Considere la siguiente situación frecuente: un script `./resultados/resumir.sh` devuelve `No such file or directory` cuando un estudiante lo ejecuta desde una terminal, mientras que en otra ventana devuelve `Permission denied`. Ante este síntoma combinado, no aplique apresuradamente `chmod +x`. El orden riguroso de diagnóstico exige:

1. **Verificar el contexto de ejecución:** Ejecutar `pwd` en ambas ventanas para comprobar si ambas sesiones se encuentran realmente en el mismo directorio de trabajo.
2. **Resolver la ruta del fichero:** Inspeccionar con `ls -ld - ./resultados/resumir.sh` para confirmar si el directorio `resultados/` o el fichero existen en esa ubicación relativa.
3. **Inspeccionar permisos:** Una vez confirmada la existencia y unicidad del fichero, examinar con `ls -l` sus bits de permiso antes de modificar nada con `chmod`.

Principio clave: Nunca intente corregir permisos sobre una ruta que no ha comprobado previamente que apunta al fichero correcto.

1.14 Síntesis de referencia

Necesidad operativa	Construcción principal	Pregunta de control
Entender el diseño	Todo es un fichero; herramientas modulares	¿Por qué combinamos herramientas pequeñas en vez de usar una sola monolítica?
Orientarse	<code>pwd, ls -la, cd</code>	¿Desde qué directorio de trabajo exacto se está interpretando esta ruta?
Organizar	<code>mkdir -p, cp, mv, touch</code>	¿He comprobado el directorio de destino antes de escribir?
Inspeccionar sin alterar	<code>file, head, tail, less, wc -l</code>	¿Qué unidad exacta estoy observando y cómo salgo de forma segura (q)?
Gestionar enlaces	<code>ln -s ORIGEN ENLACE</code>	¿Estoy evitando duplicar gigabytes de datos biológicos en el disco?
Proteger y compartir	Permisos <code>rxw</code> , <code>chmod</code> , <code>chown</code>	¿Quién tiene permiso de lectura, escritura o ejecución en este archivo o carpeta?
Ejecutar programas	<code>\$PATH</code> , ruta explícita <code>./</code> , <code>\$?</code>	¿El estado de salida (<code>\$?</code>) vale 0 indicando finalización exitosa?
Configurar entorno	Variables locales vs <code>export</code> , <code>~/.bashrc</code>	¿Esta variable de entorno se transmite a los subprocesos que lance?

Necesidad operativa	Construcción principal	Pregunta de control
Ergonomía de trabajo	Tab, Ctrl+C, Ctrl+L, Ctrl+R	¿Estoy usando el autocompletado para evitar erratas en identificadores?
Monitorizar disco	df -h, du -sh	¿Hay suficiente espacio en el sistema de archivos antes de lanzar el análisis?
Seguridad destructiva	Confirmar con pwd y ls -ld -RUTA	¿Es exactamente el objetivo previsto antes de ejecutar una eliminación?
Reproducir y entregar	tar -czf/-tzf, sha256sum -c	¿El paquete conserva la estructura, los metadatos y la integridad exacta de los bytes?

La tabla de síntesis enlaza cada necesidad operativa del capítulo con su construcción principal y una pregunta de control verificable.

1.15 Glosario esencial

- **Unix / tipo Unix:** Sistema operativo original de 1969 frente a sus descendientes y clones modernos (Linux, macOS, BSD).
- **Proceso:** Programa en ejecución en la memoria con un identificador único (PID) y recursos propios asignados.
- **Terminal:** Interfaz de texto que conecta al usuario con el intérprete de órdenes.
- **Shell:** Intérprete de comandos de línea de órdenes; en este texto, Bash.
- **Estado de salida (\$?):** Número entero (0 para éxito, mayor que 0 para error) que un proceso devuelve al finalizar.
- **\$PATH:** Lista ordenada de directorios en los que la Shell busca ejecutables por su nombre simple.
- **Directorio de trabajo (cwd):** Posición actual dentro del árbol del sistema de archivos desde donde se resuelven las rutas relativas.
- **Enlace simbólico (symlink):** Fichero especial que actúa como puntero transparente a otra ruta del sistema sin duplicar los datos físicos.
- **Permisos rwx:** Permisos de lectura (r), escritura (w) y ejecución (x) para usuario, grupo y otros.
- **Variable de entorno:** Variable exportada disponible para todos los subprocesos que la Shell engendre.
- **Reproducibilidad:** Capacidad de repetir íntegramente un análisis computacional obteniendo idénticos resultados gracias al registro explícito del entorno.
- **Checksum SHA-256:** Huella digital criptográfica que garantiza que un fichero no ha sufrido modificaciones byte a byte.

1.16 Fuentes y límites de uso

Este capítulo reconstruye y expande la base curricular del módulo legacy LEG-TC01 y, de manera primordial, las presentaciones docentes oficiales del curso impartidas por Álvaro Serrano Navarro: 20250909_intro.pptx (Módulo 0, historia, arquitectura y filosofía Unix) y 20250911_fundamentos.pptx (Módulo 1, 65 diapositivas con el modelo de procesos, gestión de permisos, enlaces, variables de entorno, comandos fundamentales y ergonomía de terminal).

El contenido se enriquece metodológicamente mediante la bibliografía de referencia de Buffalo (2015) y Allesina & Wilmes (2019), así como los manuales oficiales de GNU Bash, GNU Coreutils y GNU tar.

Anexo práctico · Ambientarse, inspeccionar y entregar de forma reproducible

Pregunta, producto y separación de materiales

¿Puede alguien que no ha visto su pantalla repetir exactamente lo que usted hizo, a partir solo de lo que entregó? Este laboratorio construye un procedimiento mínimo —explorar, registrar el entorno y empaquetar— que responde que sí.

El producto individual contiene: un registro de entorno con los siete campos mínimos de la sección de reproducibilidad de la teoría (TC01-SEC-REPRO), un `README.md` con las órdenes ejecutadas y sus supuestos, y un paquete `.tar.gz` verificado con `sha256sum` que reproduce la estructura de `datos/`, `notas/` y el propio registro.

El anexo es `own`: no comparte laboratorio con otro capítulo. Usa `practice_assets/` como espacio de trabajo de partida y `data/` como entrada de solo lectura; no modifica `data/`. La clave docente (`instructor_key/`) no se distribuye al alumnado.

Mapa de ruta y carga

Bloque	Minutos	Producto
preparar el espacio de trabajo	10–15	directorio <code>datos/notas/</code> creado sin tocar <code>data/</code>
explorar e inspeccionar	20–25	respuestas comprobadas con <code>wc/file/head/tail</code>
registrar el entorno	15–20	registro con los siete campos mínimos
protocolo antes de borrar (simulado)	10–15	nota comparando variable definida y vacía
empaquetar y verificar	15–20	<code>.tar.gz</code> + <code>sha256sum -c</code> correcto

La ruta única de este anexo (no hay separación esencial/estudio) suma 70–95 minutos y produce evidencia comprobable en cada bloque.

1 · Preparar el espacio de trabajo

Listing 1.2: Comprobación de entorno y espacio de trabajo.

```
bash --version
chmod +x practice_assets/create_workspace.sh
./practice_assets/create_workspace.sh TC01_entrega
cd TC01_entrega
```

`create_workspace.sh` crea `datos/`, `notas/` y `resultados/`, y copia (no enlaza) los cuatro ficheros de `data/` del capítulo dentro de `datos/`, conservando el nombre con espacio.

2 · Explorar e inspeccionar sin alterar

Tarea. Registre en `notas/respuestas.tsv` (columnas `pregunta`, `orden`, `resultado`): (a) cuántas líneas tiene cada uno de los cuatro ficheros de `datos/`, contrastando `wc -l` con un conteo manual del más corto; (b) qué tipo identifica `file` para `tc01_sample.log` y si coincide con lo que sugiere su extensión; (c) las 5 primeras y las 3 últimas líneas de `tc01_sample.fasta` con `head/tail`; (d) la ruta absoluta de `datos/` obtenida con `pwd` desde dentro de `TC01_entrega`.

Prediga por escrito, antes de ejecutar, cuántas líneas espera contar en `tc01_sample.fasta` a partir de lo que observe con `less` o `head`; compare después con `wc -l` y anote si coinciden.

3 · Registrar el entorno

Listing 1.3: Generar y comprobar el registro de entorno.

```
../practice_assets/check_environment_registration.sh notas/entorno.tsv
```

`check_environment_registration.sh` rechaza un registro al que le falte cualquiera de los siete campos mínimos (sistema, versión de Bash, origen de datos, fecha, órdenes, resultado esperado, limitaciones) y solo acepta uno completo. Complete manualmente los campos que el script no puede rellenar solo (origen de los datos, resultado esperado, limitaciones): son juicios suyos, no observaciones automáticas.

4 · Confirmar antes de escribir

Tarea. Antes de guardar nada en `resultados/`, ejecute `pwd` para confirmar dónde está y `ls -ld resultados/` para confirmar que el directorio existe y es el que espera. Copie ambas salidas en `notas/proteccion.txt` y escriba una frase explicando qué habría visto en `ls -ld` si el directorio no existiera todavía.

Este capítulo no invoca `rm` en ningún paso. El caso más peligroso —una variable vacía dentro de una orden destructiva— necesita variables, que TC-CH02 enseña; su anexo práctico retoma este mismo protocolo sobre ese caso concreto.

5 · Empaquetar y verificar la entrega

Listing 1.4: Empaquetar, listar y comprobar antes de entregar.

```
cd ..
tar -czf TC01_entrega.tar.gz TC01_entrega
tar -tzf TC01_entrega.tar.gz | head -n 20
sha256sum TC01_entrega.tar.gz
../practice_assets/check_delivery.sh TC01_entrega TC01_entrega.tar.gz
```

`check_delivery.sh` verifica, con Bash y GNU Coreutils exclusivamente, que el paquete contiene `datos/`, `notas/entorno.tsv` y `notas/respuestas.tsv`, que `datos/` no fue modificado respecto a `data/manifest.sha256`, y que el `.tar.gz` se extrae de forma íntegra en un directorio temporal.

Rúbrica de evaluación

La evaluación de este laboratorio pondera conjuntamente la ejecución técnica automatizada, la calidad del registro científico y la fundamentación conceptual sobre 10 puntos (reparto 30/70 según ADR-001):

- **3,0 puntos · Entrega técnica íntegra y verificador (DELIVERY_OK):** Generación correcta del paquete comprimido `.tar.gz`, estructura requerida de carpetas (`datos/`, `notas/`, `resultados/`), inalterabilidad estricta de los datos de partida y finalización exitosa de `check_delivery.sh`.
- **2,5 puntos · Registro de entorno exhaustivo y reproducible:** Completitud de los siete campos mínimos; los campos de juicio manual (origen, resultado esperado y limitaciones) son específicos, rigurosos y permiten una repetición independiente en otro equipo.
- **2,0 puntos · Exploración, predicción y resolución de rutas:** Contraste correcto entre conteos automáticos y manuales (`wc -l` vs inspección visual), resolución rigurosa de rutas relativas/absolutas y manejo adecuado de nombres con espacios.
- **1,5 puntos · Protocolo de seguridad y análisis de integridad:** Justificación adecuada del protocolo previo a operaciones en disco y comprensión explícita de los límites del checksum SHA-256 (integridad criptográfica vs validez semántica/biológica).
- **1,0 punto · Defensa razonada y contextualización:** Justificación concisa (100–150 palabras), clara y ejemplificada a partir de la propia entrega sobre los campos críticos para la reproducibilidad frente a la información contextual.

Nota didáctica: Obtener `DELIVERY_OK` es un requisito previo indispensable, pero no compensa un registro carente de rigor ni una defensa conceptualmente errónea.

Lista de autocorrección

- | | |
|--|---|
| • puedo decir desde qué directorio se interpreta cada ruta relativa que usé; | • comparé la predicción con el resultado real y anoté si coincidieron; |
| • no modifiqué ningún fichero de <code>data/</code> ; | • ninguna orden de mi entrega usa <code>rm</code> ; |
| • el nombre con espacio está siempre entrecomillado en mis notas; | • <code>check_delivery.sh</code> termina con <code>DELIVERY_OK</code> ; |
| • el registro de entorno tiene los siete campos, no solo los automáticos; | • puedo explicar la diferencia entre "el checksum coincide" y "el contenido es correcto". |

Defensa

En 100–150 palabras, explique qué campo de su registro de entorno sería imprescindible para que otra persona, en otra máquina, repitiera exactamente su exploración, y qué campo es solo información de contexto. Use un ejemplo concreto de su propia entrega, no una afirmación general.

Fuentes

Los datos de partida están en `data/` (`sources_manifest.csv`, TC01-C5-DATA, CC0-1.0). El guion de protocolo de seguridad se apoya en Buffalo (2015), pp. 67–70, ya citado en `theory.tex`.

Bibliography

- [1] Vince Buffalo. *Bioinformatics Data Skills: Reproducible and Robust Research with Open Source Tools*. O'Reilly Media, 2015. Pasajes comprobados en las páginas físicas 51–54 y 67–70 del PDF local.
- [2] GNU Project. *Bash Reference Manual*, 2026. Consultado el 6 de agosto de 2026. <https://www.gnu.org/software/bash/manual/bash.pdf>.
- [3] GNU Project. *GNU tar Manual*, 2026. Consultado el 6 de agosto de 2026. <https://www.gnu.org/software/tar/manual/tar.html>.